
```
>>> spam = {'name': 'Zophie'}
>>> spam['name']
'Zophie'
```

Although list indexes are limited to integers, a Python dictionary's index operator is a *key* and can be any hashable object. A *hash* is an integer that acts as a sort of fingerprint for a value. An object's hash never changes for the lifetime of the object, and objects with the same value must have the same hash. The string 'name' in this instance is the key for the value 'Zophie'. The `hash()` function will return an object's hash if the object is *hashable*. Immutable objects, such as strings, integers, floats, and tuples, can be hashable. Lists (as well as other mutable objects) aren't hashable. Enter the following into the interactive shell:

```
>>> hash('hello')
-1734230105925061914
>>> hash(42)
42
>>> hash(3.14)
322818021289917443
>>> hash((1, 2, 3))
2528502973977326415
>>> hash([1, 2, 3])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Although the details are beyond the scope of this book, the key's hash is used to find items stored in a dictionary and set data structures. That's why you can't use a mutable list for a dictionary's keys:

```
>>> d = {}
>>> d[[1, 2, 3]] = 'some value'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

A hash is different from an identity. Two different objects with the same value will have different identities but the same hash. For example, enter the following into the interactive shell: